

Real Numbers

Overview

Real numbers can be represented in binary using a few key techniques:

- converting denary values to binary
 - handling repeating decimals
 - using two's complement for signed values
 - storing values using mantissa and exponent form
-

1. Denary to Binary Conversion

To convert a real number to binary:

1. Convert the integer part to binary
2. Multiply the fractional part by 2
3. Record the integer part as the next binary bit
4. Repeat until the fractional part becomes 0 or a repeating cycle is detected

Example: 7.25

The integer part is 7:

- 7 in binary = 111

The fractional part is 0.25:

- $0.25 \times 2 = 0.5 \rightarrow \text{bit} = 0$
- $0.5 \times 2 = 1.0 \rightarrow \text{bit} = 1$

So:

- $7.25 = 111.01$

4	2	1	.	1/2	1/4	1/8
1	1	1	.	0	1	0

Result:

- Binary value = 111.01
-

2. Repeating Decimals

If a decimal fraction keeps repeating, stop after one full cycle.

- 10 in binary = 1010
- fractional part: 0.7

Step	Calculation	Bit
1	$0.7 \times 2 = 1.4$	1
2	$0.4 \times 2 = 0.8$	0
3	$0.8 \times 2 = 1.6$	1
4	$0.6 \times 2 = 1.2$	1
5	$0.2 \times 2 = 0.4$	0
6	$0.4 \times 2 = 0.8$	0 → repeats

Therefore:

- $10.7 \approx 1010.101100$

The repeating section is ignored after one cycle, so the answer is shown as a recurring binary pattern.

3. Two's Complement

Two's complement is used to represent signed numbers in binary, especially when handling floating-point mantissas.

Method 1: Flip & Add 1

1. Flip every bit

2. + 1

Method 2: Place-Value Table (*method used in the examples below*)

Same as normal binary place values, **except the leftmost (Most Significant) bit is negative.**

-128	64	32	16	8	4	2	1
------	----	----	----	---	---	---	---

Convert -47 to 8-bit Two's Complement

-128	64	32	16	8	4	2	1
1	1	0	1	0	0	0	1

$$-128 + 64 + 16 + 1 = -47 \rightarrow 11010001$$

4. Mantissa and Exponent Method

Computers cannot store decimals directly, so real numbers are stored as:

- a mantissa
- an exponent

This is the method used in floating-point representation.

Mantissa

The mantissa contains the significant digits of the number.

- It represents the precision of the value
- More digits usually means greater precision

Example:

- 5.25
- 5.2575

The second value has more precision because it contains more digits after the decimal point.

Exponent

The exponent shows how far the decimal point moves.

- It represents the range of the number
- It is related to the position of the decimal point

Convert +47.25

We use:

- 12-bit mantissa
- 5-bit exponent

First, convert 47.25 to binary:

-128	64	32	16	8	4	2	1	.	1/2	1/4	1/8	1/16
0	0	1	0	1	1	1	1	.	0	1	0	0

This gives:

- $47.25 = 101111.01$ in binary

Final Mantissa

Move the decimal point so it is just before the first 1:

0	.	1	0	1	1	1	1	0	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---

The decimal point moves 6 places to the left, so the exponent is 6.

Final exponent

-16	8	4	2	1
0	0	1	0	0

So the value is represented using:

- mantissa = 0.10111101000
 - exponent = 6
-

5. Normalization

Why it is needed

Floating-point numbers cannot always be stored exactly because the computer has a limited number of bits.

- $1/3 = 0.333333333...$
- A stored value is usually slightly higher or lower than the real value
- This is called rounding error

Examples:

- $0.1 + 0.1 + 0.1 = 0.2999999999999999...$
- $0.1 + 0.1 + 0.1 = 0.30000000000000001...$

Normalized form

Normalization reduces error and makes the representation more accurate.

The important rule is:

- the first two bits of the mantissa cannot be the same
- valid patterns are: 01 or 10
- invalid patterns are: 00 or 11

Simple diagram

Normalized	Meaning
01	valid
10	valid
00	not normalized
11	not normalized

This means the mantissa should start with a different pair of bits, not the same pair.

Why normalization matters

- keeps the mantissa accurate

- avoids losing bits
- removes multiple representations of the same number
- helps represent very large and very small numbers more reliably

Normalization is basically the process of adjusting the decimal point so the number is stored in the most efficient and accurate format.

Example: Normalize a mantissa

Before normalization

0	.	0	0	1	1	1	1	0
---	---	---	---	---	---	---	---	---

This starts with 00, which is not normalized.

Correct position for the decimal point

- Move the decimal point to the left until the mantissa begins with 01 or 10
- This is the normalized form

0	.	1	0	1	1	1	1	0
---	---	---	---	---	---	---	---	---

Exponent example

-128	64	32	16	8	4	2	1
0	0	0	1	1	0	0	0

This value is 24 in binary, so the exponent is adjusted by the number of places the decimal point moved.

Final rule

- Move the decimal point between the first two bits of the mantissa
 - Adjust the exponent by the same number of places moved
 - If you move left, exponent increases
 - If you move right, exponent decreases
-

Quick Summary

- Denary numbers are converted to binary by splitting integer and fractional parts
- Repeating decimals are stopped after one cycle
- Two's complement is used for signed integer values
- Mantissa and exponent store real numbers in a compact form
- Normalization keeps the number accurate and avoids wasted or repeated representations